

Module 8: Stored Procedures and Functions in PL/SQL

Duration

8–10 Hours (Theory + Laboratory Practice)

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of modular programming.
 - Explain the advantages of stored procedures and functions.
 - Create, compile, execute, modify, and delete stored procedures.
 - Pass parameters using IN, OUT, and IN OUT modes.
 - Create and use stored functions.
 - Differentiate between procedures and functions.
 - Develop reusable database programs.
 - Apply procedures and functions in real-world database applications.
-

8.1 Introduction to Stored Programs

What is a Stored Program?

A **Stored Program** is a PL/SQL program unit that is compiled and permanently stored inside the Oracle database. Instead of writing the same code repeatedly, we create it once and execute it whenever required.

Stored programs improve:

- Code Reusability
 - Performance
 - Security
 - Maintainability
-

Types of Stored Programs

Oracle supports several stored program units.

Stored Program Purpose

Procedure	Performs a task
Function	Returns a value
Package	Collection of Procedures and Functions
Trigger	Executes automatically on database events

In this module, we will study **Procedures** and **Functions**.

Real-Life Analogy

Imagine a calculator.

Whenever you press:

Addition

Subtraction

Multiplication

Division

The calculator does not rewrite the logic every time.

Instead,

it simply calls the stored operation.

Similarly,

PL/SQL stores procedures and functions inside the database.

Why Do We Need Stored Procedures?

Suppose we want to calculate employee bonuses.

Without procedures,

the calculation must be written in every program.

Program 1

Bonus Logic

Program 2

Bonus Logic

Program 3

Bonus Logic

The same code is repeated.

Instead,

Create Procedure Once

↓

Call Procedure Whenever Needed

Advantages

- Reusable code
- Reduced redundancy
- Faster execution
- Easier maintenance
- Better security
- Centralized business logic

8.2 Stored Procedure

What is a Procedure?

A **Procedure** is a named PL/SQL block stored in the database that performs a specific task.

A procedure may or may not accept parameters and does not necessarily return a value.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
```

```
IS
```

```
BEGIN
```

```
    statements;
```

```
END;
```

```
/
```

Example 1: Simple Procedure

```
CREATE OR REPLACE PROCEDURE greet
```

```
IS
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');
```

```
END;
```

```
/
```

```
Execute
```

```
BEGIN
```

```
greet;
```

```
END;  
/  
Output  
Welcome to PL/SQL
```

How a Procedure Works

Create Procedure

↓

Stored in Database

↓

Call Procedure

↓

Execute Statements

↓

Return Control

Example 2: Employee Details

```
CREATE OR REPLACE PROCEDURE employee_info  
IS  
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('Employee Name : Rahul');
```

```
DBMS_OUTPUT.PUT_LINE('Department : IT');
```

```
END;
```

```
/
```

Execution

```
BEGIN
```

```
employee_info;
```

```
END;
```

```
/
```

8.3 Parameters in Procedures

A procedure becomes more useful when it accepts input from users.

Oracle provides three parameter modes.

Parameter Mode Purpose

IN	Input only
OUT	Output only
IN OUT	Input and Output

IN Parameter

An IN parameter passes data from the calling program to the procedure.

Syntax

```
CREATE PROCEDURE procedure_name
```

```
(parameter datatype)
```

```
IS
```

```
BEGIN
```

```
...
```

```
END;
```

Example

```
CREATE OR REPLACE PROCEDURE display_name
```

```
(p_name VARCHAR2)
```

```
IS
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('Name : '||p_name);
```

```
END;
```

```
/
```

```
Execution
```

```
BEGIN
```

```
display_name('Amit');
```

```
END;
```

```
/
```

```
Output
```

```
Name : Amit
```

Example: Area of Circle

```
CREATE OR REPLACE PROCEDURE area_circle
```

```
(radius NUMBER)
```

```
IS
```

```
area NUMBER;
```

```
BEGIN
```

```
area:=3.14159*radius*radius;
```

```
DBMS_OUTPUT.PUT_LINE(area);
```

```
END;
```

```
/
```

OUT Parameter

OUT parameters send values back to the calling program.

Example

CREATE OR REPLACE PROCEDURE square

(num IN NUMBER,
result OUT NUMBER)

IS

BEGIN

result:=num*num;

END;

/

Execution

DECLARE

ans NUMBER;

BEGIN

square(5,ans);

DBMS_OUTPUT.PUT_LINE(ans);

END;

/

Output

25

IN OUT Parameter

IN OUT parameters receive a value, modify it, and return the updated value.

Example

CREATE OR REPLACE PROCEDURE increment_salary

(salary IN OUT NUMBER)

IS

BEGIN

salary:=salary+5000;

END;

/

Execution

DECLARE

emp_salary NUMBER:=45000;

BEGIN

increment_salary(emp_salary);

```
DBMS_OUTPUT.PUT_LINE(emp_salary);
```

```
END;
```

```
/
```

```
Output
```

```
50000
```

8.4 Stored Function

What is a Function?

A **Function** is a stored PL/SQL program that always returns exactly one value using the RETURN statement. Unlike procedures, functions are often used in SQL expressions and calculations.

Syntax

```
CREATE FUNCTION function_name
```

```
RETURN datatype
```

```
IS
```

```
BEGIN
```

```
RETURN value;
```

```
END;
```

```
/
```

Example

```
CREATE OR REPLACE FUNCTION addition
```

```
(a NUMBER,
```

```
b NUMBER)
```

```
RETURN NUMBER
```

```
IS
```

```
BEGIN
```

```
RETURN a+b;
```

```
END;
```

```
/
```

```
Execution
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE(addition(20,30));
```

```
END;
```

```
/
```

```
Output
```

```
50
```

Example: Calculate GST

```
CREATE OR REPLACE FUNCTION calculate_gst
```

(amount NUMBER)

RETURN NUMBER

IS

BEGIN

RETURN amount*0.18;

END;

/

Execution

BEGIN

DBMS_OUTPUT.PUT_LINE(calculate_gst(10000));

END;

/

Output

1800

Example: Factorial Function

CREATE OR REPLACE FUNCTION factorial

(n NUMBER)

RETURN NUMBER

IS

fact NUMBER:=1;

BEGIN

FOR i IN 1..n LOOP

fact:=fact*i;

END LOOP;

RETURN fact;

END;

/

Execution

BEGIN

DBMS_OUTPUT.PUT_LINE(factorial(5));

END;

/

Output

120

Calling Functions in SQL

Unlike procedures,
functions can be used inside SQL statements.

Example

```
SELECT addition(100,200)
```

```
FROM dual;
```

Output

300

Procedure vs Function

Procedure

Performs a task

Return statement optional

Can have IN, OUT, IN OUT parameters

Cannot normally be used directly in SQL expressions

Invoked using EXEC or PL/SQL block

Function

Returns a value

RETURN statement mandatory

Normally uses IN parameters and returns one value

Can be used in SQL expressions (subject to Oracle rules)

Can be invoked from SQL or PL/SQL

Creating, Replacing and Dropping

Replace

```
CREATE OR REPLACE PROCEDURE
```

Updates an existing procedure without dropping it first.

Delete Procedure

```
DROP PROCEDURE procedure_name;
```

Delete Function

```
DROP FUNCTION function_name;
```

Advantages of Stored Procedures and Functions

- Modular programming
- Code reusability
- Faster execution due to compilation
- Reduced network traffic
- Enhanced database security
- Centralized business logic
- Easier maintenance
- Simplified debugging and testing

Real-World Applications

Stored procedures and functions are extensively used in:

- Banking Systems
- Payroll Processing
- Hospital Management
- University ERP Systems
- E-Commerce Platforms
- Insurance Claim Processing
- Railway Reservation Systems
- Inventory Management
- Financial Reporting
- Online Examination Systems

Practical Programs

Program 1

Create a procedure to display student details.

Program 2

Create a procedure to calculate the area of a rectangle.

Program 3

Create a procedure to swap two numbers using IN OUT parameters.

Program 4

Create a function to determine whether a number is even or odd.

CREATE OR REPLACE FUNCTION is_even

(n NUMBER)

RETURN VARCHAR2

IS

BEGIN

IF MOD(n,2)=0 THEN

RETURN 'Even';

ELSE

RETURN 'Odd';

END IF;

END;

/

Program 5

Create a function to calculate income tax based on salary.

Program 6

Create a function to calculate the average marks of five subjects.

Program 7

Create a procedure to update an employee's salary based on employee ID.

Program 8

Create a function to determine the largest of three numbers.

Best Practices

- Use meaningful names for procedures and functions.
 - Validate input parameters before processing.
 - Include exception handling in stored programs.
 - Avoid duplicating business logic across multiple procedures.
 - Use comments to improve readability and maintainability.
 - Test procedures and functions with different input values.
-

Common Errors by Beginners

- Forgetting the RETURN statement in a function.

- Returning a value of the wrong data type.
 - Using an OUT parameter without assigning a value.
 - Passing parameters in the wrong order.
 - Omitting the terminating / after creating a stored program in SQL*Plus or SQL Developer.
-

Summary

This module introduced **Stored Procedures and Functions**, two essential components of modular programming in PL/SQL. Students learned how to create, execute, modify, and delete procedures and functions, pass parameters using IN, OUT, and IN OUT modes, and distinguish between procedures and functions. These reusable program units improve performance, maintainability, and code organization, making them indispensable in enterprise database applications.

Laboratory Exercises

1. Create a procedure to display employee details.
 2. Create a procedure that accepts two numbers and displays their sum.
 3. Create a procedure to calculate the area of a circle.
 4. Create a procedure using an OUT parameter to calculate the square of a number.
 5. Create a procedure using an IN OUT parameter to increment an employee's salary.
 6. Create a function to calculate the factorial of a number.
 7. Create a function to check whether a number is prime.
 8. Create a function to calculate GST for a given purchase amount.
 9. Create a function to find the largest among three numbers.
 10. Create a procedure to update an employee's salary and handle exceptions appropriately.
-

Viva/Interview Questions

1. What is a stored procedure?
2. What is a stored function?
3. Differentiate between a procedure and a function.
4. Explain the purpose of IN, OUT, and IN OUT parameters.
5. Why is the RETURN statement mandatory in a function?
6. Can a function be used inside an SQL query? Explain with an example.
7. What is the purpose of CREATE OR REPLACE?
8. How do you delete a stored procedure or function?
9. What are the advantages of stored procedures and functions in enterprise applications?
10. Give two real-world scenarios where procedures and functions are commonly used.